

Seance 2

Heritage et Polymorphisme

Le pouvoir de la reutilisation et de la flexibilite

Cours de renforcement C++

Table des matières

1	Introduction : Le probleme de la duplication	3
1.1	Un exemple concret : Des animaux	3
1.2	La solution : L'heritage	3
2	L'heritage : Comment ca marche ?	6
2.1	La syntaxe de base	6
2.2	Que recoit la classe derivee ?	6
2.3	L'ordre de construction et destruction	7
2.4	Appeler le constructeur de la classe de base	7
3	Le polymorphisme : La vraie magie	8
3.1	Le probleme sans polymorphisme	8
3.2	La solution : Le mot-cle virtual	8
3.3	La vtable : Comment ca marche en coulisse ?	8
3.4	Override : Verifier qu'on redefinit bien	9
3.5	Final : Interdire la redefinition	9
4	Destructeur virtuel : Obligatoire !	11
4.1	Le piege mortel	11
4.2	La solution : Virtual destructeur	11
5	Heritage vs Composition : Quand utiliser quoi ?	13
5.1	Heritage : La relation "EST-UN"	13
5.2	Composition : La relation "A-UN"	13
5.3	Exemple : Mauvais usage de l'heritage	14
5.4	Preferer la composition a l'heritage	15
6	Exemples pratiques complets	16
6.1	Exemple 1 : Systeme de formes geometriques	16
6.2	Exemple 2 : Systeme de paiement	18
7	Exercices pratiques	21
7.1	Exercice 1 : Hierarchie d'employes (facile)	21
7.2	Exercice 2 : Zoo virtuel (moyen)	22
7.3	Exercice 3 : Calculatrice polymorphe (difficile)	24
8	Pieges courants	27
8.1	Piege 1 : Oublier virtual sur le destructeur	27
8.2	Piege 2 : Slicing (decoupe)	27
8.3	Piege 3 : Appeler une fonction virtuelle dans le constructeur	27

9 Quiz de validation	29
9.1 Questions	29
9.2 Reponses	29
10 Pour aller plus loin	30
10.1 Heritage multiple	30
10.2 Classes abstraites pures	30
10.3 Exercices bonus	30
10.4 Ressources supplementaires	31

1 Introduction : Le probleme de la duplication

1.1 Un exemple concret : Des animaux

Imaginez que vous devez modeliser differents animaux dans un jeu video.

Sans heritage (mauvais) :

```

1  class Chien {
2      std::string nom;
3      int age;
4  public:
5      void manger() { std::cout << "Le chien mange\n"; }
6      void dormir() { std::cout << "Le chien dort\n"; }
7      void aboyer() { std::cout << "Ouaf ouaf !\n"; }
8  };
9
10 class Chat {
11     std::string nom; // Duplication !
12     int age;         // Duplication !
13 public:
14     void manger() { std::cout << "Le chat mange\n"; } // Duplication
15     !
16     void dormir() { std::cout << "Le chat dort\n"; } // Duplication
17     !
18     void miauler() { std::cout << "Miaou !\n"; }
19 };
20
21 class Oiseau {
22     std::string nom; // Encore de la duplication !
23     int age;         // Encore de la duplication !
24 public:
25     void manger() { std::cout << "L'oiseau mange\n"; } // Encore !
26     void dormir() { std::cout << "L'oiseau dort\n"; } // Encore !
27     void voler() { std::cout << "L'oiseau vole !\n"; }
28 };

```

Les problemes

- **Code duplique partout** : nom, age, manger(), dormir()
- **Difficile a maintenir** : Si vous voulez changer manger(), il faut le changer dans 3 classes!
- **Pas d'unite** : Impossible de traiter "tous les animaux" de la meme facon

1.2 La solution : L'heritage

Avec heritage (bon) :

```

1  // Classe de BASE (ou classe MERE ou classe PARENTE)
2  class Animal {
3  protected:
4      std::string nom;
5      int age;
6
7  public:
8      Animal(const std::string& n, int a) : nom(n), age(a) {}
9
10     void manger() {

```

```

11     std::cout << nom << " mange\n";
12 }
13
14 void dormir() {
15     std::cout << nom << " dort\n";
16 }
17 };
18
19 // Classes DERIVEES (ou classes FILLES ou classes ENFANTS)
20 class Chien : public Animal {
21 public:
22     Chien(const std::string& n, int a) : Animal(n, a) {}
23
24     void aboyer() {
25         std::cout << nom << " fait Ouaf ouaf !\n";
26     }
27 };
28
29 class Chat : public Animal {
30 public:
31     Chat(const std::string& n, int a) : Animal(n, a) {}
32
33     void miauler() {
34         std::cout << nom << " fait Miaou !\n";
35     }
36 };
37
38 class Oiseau : public Animal {
39 public:
40     Oiseau(const std::string& n, int a) : Animal(n, a) {}
41
42     void voler() {
43         std::cout << nom << " vole !\n";
44     }
45 };

```

Utilisation :

```

1 int main() {
2     Chien rex("Rex", 5);
3     Chat felix("Felix", 3);
4     Oiseau tweety("Tweety", 2);
5
6     // Tout le monde peut manger et dormir (herite de Animal)
7     rex.manger();      // Rex mange
8     felix.dormir();    // Felix dort
9     tweety.manger();   // Tweety mange
10
11     // Chacun a ses specificites
12     rex.aboyer();      // Rex fait Ouaf ouaf !
13     felix.miauler();   // Felix fait Miaou !
14     tweety.voler();    // Tweety vole !
15 }

```

Les avantages

- **Zero duplication** : Le code commun est dans Animal
- **Facile a maintenir** : Changer manger() une seule fois
- **Extensible** : Ajouter un nouveau animal est facile
- **Reutilisable** : On peut traiter tous les animaux uniformement

2 L'heritage : Comment ca marche ?

2.1 La syntaxe de base

```

1 class ClasseDerivee : public ClasseDeBase {
2     // Contenu de la classe derivee
3 };

```

Vocabulaire

- **ClasseDeBase** : aussi appelee classe mere, classe parente, superclasse
- **ClasseDerivee** : aussi appelee classe fille, classe enfant, sous-classe
- **Heriter** : recuperer les membres (variables et methodes) de la classe de base
- **public** : le type d'heritage (nous verrons pourquoi plus tard)

2.2 Que recoit la classe derivee ?

Une classe derivee herite de TOUT ce qui est public ou protected de la classe de base.

```

1 class Vehicule {
2     private:
3         double carburant; // PRIVE : pas herite (mais existe quand meme)
4
5     protected:
6         int vitesse;      // PROTEGE : herite et accessible dans les classes
7                             // filles
8
9     public:
10        std::string marque; // PUBLIC : herite et accessible partout
11
12        void demarrer() {
13            std::cout << "Le vehicule démarre\n";
14        }
15 };
16
17 class Voiture : public Vehicule {
18     public:
19         void accelerer() {
20             vitesse += 10; // OK : vitesse est protected
21             // carburant -= 0.1; // ERREUR ! carburant est private
22             std::cout << marque << " accelere\n"; // OK : marque est public
23         }
24 };

```

Protected : Le juste milieu

private : Accessible uniquement dans la classe elle-meme
protected : Accessible dans la classe ET dans les classes derivees
public : Accessible partout
Conseil : Utilisez protected pour les membres que les classes derivees doivent pouvoir modifier.

2.3 L'ordre de construction et destruction

Question importante : Dans quel ordre les constructeurs sont-ils appeles ?

```

1 class Base {
2 public:
3     Base() { std::cout << "1. Constructeur Base\n"; }
4     ~Base() { std::cout << "4. Destructeur Base\n"; }
5 };
6
7 class Derivee : public Base {
8 public:
9     Derivee() { std::cout << "2. Constructeur Derivee\n"; }
10    ~Derivee() { std::cout << "3. Destructeur Derivee\n"; }
11 };
12
13 int main() {
14     std::cout << "Creation :\n";
15     Derivee d;
16     std::cout << "Destruction :\n";
17 }
18
19 // Affiche :
20 // Creation :
21 // 1. Constructeur Base
22 // 2. Constructeur Derivee
23 // Destruction :
24 // 3. Destructeur Derivee
25 // 4. Destructeur Base

```

Construction : De la base vers la derivee (comme construire les fondations avant les murs)

Destruction : De la derivee vers la base (ordre inverse, comme demolir les murs avant les fondations)

Logique : La classe derivee peut utiliser la classe de base, donc la base doit exister en premier !

2.4 Appeler le constructeur de la classe de base

```

1 public Personne(const stdstring n, int a) nom(n), age(a) stdcout << "Personne creee
  " << nom << " " ;
2 class Etudiant : public Personne private: std::string ecole;
3 public: // IMPORTANT : On appelle le constructeur de Personne Etudiant(const
  std::string n, int a, const std::string e) : Personne(n, a), ecole(e) std::cout <<
  "Etudiant inscrit a " << ecole << " ";
4 void afficher() std::cout << nom << " " << age << " ans, etudie a " << ecole << " " ;
5 int main() Etudiant alice("Alice", 20, "Epitech"); alice.afficher();
6 // Affiche : // Personne creee : Alice // Etudiant inscrit a Epitech // Alice, 20 ans,
  etudie a Epitech

```

Q

3 Le polymorphisme : La vraie magie

3.1 Le probleme sans polymorphisme

Imaginons un zoo avec différents animaux. Vous voulez tous les faire parler.

Sans polymorphisme (mauvais) :

```

7 public void parler()  stdout << "..." // Son generique
8 class Chien public Animal public void parler()  stdout << "Ouaf "
9 class Chat public Animal public void parler()  stdout << "Miaou "
10 int main()  Chien rex Chat felix
11 rex.parler() // Ouaf  felix.parler() // Miaou
12 // Maintenant, stockons-les dans un tableau Animal* animaux[2] animaux[0] = rex //
   Pointeur vers Chien animaux[1] = felix // Pointeur vers Chat
13 // Faisons-les tous parler for (int i = 0 i < 2 ++i)  animaux[i]->parler()
14 // Affiche // ... // ... // <- PROBLEME  Appelle Animalparler(), pas
   Chien/Chatparler()
15 ::::::::::::::!!

```

3.2 La solution : Le mot-cle virtual

Avec polymorphisme (bon) :

```

16 // IMPORTANT Destructeur virtuel obligatoire virtual Animal() = default ;
17 class Chien : public Animal public: void parler() override // override : verifie
    qu'on redefinit bien std::cout < "Ouaf !"; ;
18 class Chat : public Animal public: void parler() override std::cout < "Miaou !"; ;
19 int main() Chien rex; Chat felix;
20 Animal* animaux[2]; animaux[0] = rex; animaux[1] = felix;
21 for (int i = 0; i < 2; ++i) animaux[i]->parler();
22 // Affiche : // Ouaf ! // Miaou ! // <- MAGIQUE ! Ca marche maintenant !

```

3.3 La vtable : Comment ça marche en coulisse ?

Quand vous mettez **virtual** sur une fonction, C++ cree une table cachee appelee **vtable** (virtual table).

Chaque objet a un pointeur cache vers sa vtable. Quand vous appelez une fonction virtuelle, C++ :

1. Regarde dans la vtable de l'objet
2. Trouve l'adresse de la bonne fonction
3. Appelle cette fonction

Le cout : Un tout petit peu plus lent (une indirection), mais ca vaut largement le coup pour la flexibilite!

3.4 Override : Verifier qu'on redefinit bien

```

1 class Animal {
2 public:
3     virtual void parler() const {
4         std::cout << "...\\n";
5     }
6 };
7
8 class Chien : public Animal {
9 public:
10     // Sans override
11     void parler() { // ERREUR subtile : on a oublie "const" !
12         std::cout << "Ouaf !\\n";
13     }
14     // Cette fonction ne redefinit PAS Animal::parler()
15     // car la signature est differente (pas de const)
16 };
17
18 class Chat : public Animal {
19 public:
20     // Avec override
21     void parler() override { // ERREUR de compilation !
22         std::cout << "Miaou !\\n";
23     }
24     // Le compilateur dit : "Tu veux redefinir, mais il n'y a
25     // aucune fonction parler() sans const dans Animal !"
26 };

```

Conseil d'or

TOUJOURS utiliser **override** quand vous redefinissez une fonction virtuelle!
Le compilateur verifiera que vous redefinissez vraiment une fonction existante, et pas que vous en creez une nouvelle par accident.

3.5 Final : Interdire la redefinition

```

1 class Animal {
2 public:
3     virtual void parler() {
4         std::cout << "...\\n";
5     }
6 };
7
8 class Chien : public Animal {
9 public:
10     // On redefinit ET on interdit toute nouvelle redefinition
11     void parler() override final {
12         std::cout << "Ouaf !\\n";
13     }
14 };
15
16 class ChienDeChasse : public Chien {

```

```
17 public:
18     // ERREUR de compilation !
19     void parler() override {
20         std::cout << "Woof !\n";
21     }
22     // Le compilateur dit : "Chien::parler() est final,
23     // tu ne peux pas la redefinir !"
24 };
```

Utilite : Parfois, vous voulez qu'une implementation soit definitive et ne puisse plus etre changee. C'est rare, mais utile pour la securite ou les performances.

4 Destructeur virtuel : Obligatoire !

4.1 Le piege mortel

```

1 class Animal {
2 public:
3     Animal() { std::cout << "Animal construit\n"; }
4     ~Animal() { std::cout << "Animal detruit\n"; }
5     // Pas de "virtual" sur le destructeur !
6 };
7
8 class Chien : public Animal {
9 private:
10     int* data;
11 public:
12     Chien() {
13         data = new int[100];
14         std::cout << "Chien construit\n";
15     }
16     ~Chien() {
17         delete[] data;
18         std::cout << "Chien detruit\n";
19     }
20 };
21
22 int main() {
23     Animal* ptr = new Chien();
24     delete ptr; // CATASTROPHE !
25 }
26
27 // Affiche :
28 // Animal construit
29 // Chien construit
30 // Animal detruit
31 // <- FUITE MEMOIRE ! ~Chien() n'est jamais appele !

```

Le probleme

Sans `virtual` sur le destructeur, seul le destructeur de la classe de base est appele !
Le destructeur de `Chien` n'est JAMAIS appele, donc la memoire allouee dans `Chien` n'est jamais liberee.

Resultat : FUITE MEMOIRE massive !

4.2 La solution : Virtual destructeur

```

1 class Animal {
2 public:
3     Animal() { std::cout << "Animal construit\n"; }
4     virtual ~Animal() { // <- virtual ici !
5         std::cout << "Animal detruit\n";
6     }
7 };
8
9 class Chien : public Animal {
10 private:
11     int* data;

```

```
12 public:
13     Chien() {
14         data = new int[100];
15         std::cout << "Chien construit\n";
16     }
17     ~Chien() override { // override optionnel mais recommande
18         delete[] data;
19         std::cout << "Chien detruit\n";
20     }
21 };
22
23 int main() {
24     Animal* ptr = new Chien();
25     delete ptr;
26 }
27
28 // Affiche :
29 // Animal construit
30 // Chien construit
31 // Chien detruit <- Appele maintenant !
32 // Animal detruit
```

Regle absolue

DES que vous avez une fonction virtuelle dans une classe, le destructeur **DOIT** etre virtuel !

En pratique : DES qu'une classe est destinee a etre une classe de base, mettez :

```
1 virtual ~MaClasse() = default;
```

= default dit au compilateur : "genere le destructeur par default, mais rends-le virtuel".

5 Heritage vs Composition : Quand utiliser quoi ?

5.1 Heritage : La relation "EST-UN"

Utilisez l'héritage quand : "X EST-UN Y"

Exemples :

- Un Chien EST-UN Animal → Heritage
- Une Voiture EST-UN Vehicule → Heritage
- Un Etudiant EST-UNE Personne → Heritage

```
1 class Vehicule {
2     // ...
3 };
4
5 class Voiture : public Vehicule {
6     // Une Voiture EST-UN Vehicule
7     // Elle herite de tout ce qu'un Vehicule peut faire
8 };
```

5.2 Composition : La relation "A-UN"

Utilisez la composition quand : "X A-UN Y"

Exemples :

- Une Voiture A-UN Moteur → Composition
- Une Personne A-UN Coeur → Composition
- Un Ordinateur A-UN Processeur → Composition

```
1 class Moteur {
2     int puissance;
3 public:
4     void demarrer() { /* ... */ }
5 };
6
7 class Voiture {
8 private:
9     Moteur moteur; // Composition : la Voiture CONTIENT un Moteur
10    std::string marque;
11
12 public:
13     void demarrer() {
14         moteur.demarrer(); // Delegation au moteur
15     }
16 };
```

5.3 Exemple : Mauvais usage de l'héritage

Mauvais exemple

```
1 // MAUVAIS : Un carre n'EST PAS un rectangle !
2 // (meme si mathematiquement ca peut se defendre)
3 class Rectangle {
4     int largeur, hauteur;
5 public:
6     void setLargeur(int l) { largeur = l; }
7     void setHauteur(int h) { hauteur = h; }
8 };
9
10 class Carre : public Rectangle {
11     // Probleme : un carre doit avoir largeur == hauteur
12     // Mais on herite de setLargeur et setHauteur
13     // qui permettent de les mettre differents !
14 };
```

Bonne solution

```
1 // BON : Carre et Rectangle sont des formes separees
2 class Forme {
3 public:
4     virtual double aire() const = 0;
5     virtual ~Forme() = default;
6 };
7
8 class Rectangle : public Forme {
9     int largeur, hauteur;
10 public:
11     double aire() const override {
12         return largeur * hauteur;
13     }
14 };
15
16 class Carre : public Forme {
17     int cote;
18 public:
19     double aire() const override {
20         return cote * cote;
21     }
22 };
```

5.4 Preferer la composition a l'heritage

Principe de conception

En cas de doute, preferez la composition !

L'heritage cree un couplage fort : La classe derivee depend fortement de la classe de base. Si vous changez la base, toutes les classes derivees sont affectees.

La composition est plus flexible : Vous pouvez changer les composants facilement sans affecter le reste.

Regle pratique :

- Heritage : pour modeliser des relations naturelles (Animal/Chien)
- Composition : pour assembler des fonctionnalites (Voiture/Moteur)

6 Exemples pratiques complets

6.1 Exemple 1 : Systeme de formes geometriques

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4
5 class Forme {
6 protected:
7     std::string couleur;
8
9 public:
10     Forme(const std::string& c) : couleur(c) {}
11     virtual ~Forme() = default;
12
13     // Methodes virtuelles pures (interface)
14     virtual double aire() const = 0;
15     virtual double perimetre() const = 0;
16     virtual void afficher() const = 0;
17
18     // Methode commune
19     std::string getCouleur() const { return couleur; }
20 };
21
22 class Cercle : public Forme {
23 private:
24     double rayon;
25
26 public:
27     Cercle(double r, const std::string& c)
28         : Forme(c), rayon(r) {}
29
30     double aire() const override {
31         return 3.14159 * rayon * rayon;
32     }
33
34     double perimetre() const override {
35         return 2 * 3.14159 * rayon;
36     }
37
38     void afficher() const override {
39         std::cout << "Cercle " << couleur
40             << " (rayon=" << rayon << ")\n";
41     }
42 };
43
44 class Rectangle : public Forme {
45 private:
46     double largeur, hauteur;
47
48 public:
49     Rectangle(double l, double h, const std::string& c)
50         : Forme(c), largeur(l), hauteur(h) {}
51
52     double aire() const override {
53         return largeur * hauteur;
54     }
55 }
```

```

55
56     double perimetre() const override {
57         return 2 * (largeur + hauteur);
58     }
59
60     void afficher() const override {
61         std::cout << "Rectangle " << couleur
62             << " (" << largeur << "x" << hauteur << ")\n";
63     }
64 };
65
66 class Triangle : public Forme {
67 private:
68     double base, hauteur;
69
70 public:
71     Triangle(double b, double h, const std::string& c)
72         : Forme(c), base(b), hauteur(h) {}
73
74     double aire() const override {
75         return (base * hauteur) / 2;
76     }
77
78     double perimetre() const override {
79         // Simplifie : on suppose un triangle isocèle
80         double cote = std::sqrt((base/2)*(base/2) + hauteur*hauteur);
81         return base + 2 * cote;
82     }
83
84     void afficher() const override {
85         std::cout << "Triangle " << couleur
86             << " (base=" << base << ", h=" << hauteur << ")\n";
87     }
88 };
89
90 int main() {
91     // Polymorphisme : vector de pointeurs vers la classe de base
92     std::vector<Forme*> formes;
93
94     formes.push_back(new Cercle(5.0, "rouge"));
95     formes.push_back(new Rectangle(4.0, 3.0, "bleu"));
96     formes.push_back(new Triangle(6.0, 4.0, "vert"));
97
98     std::cout << "=== Affichage des formes ===\n";
99     for (Forme* f : formes) {
100         f->afficher();
101         std::cout << "   Aire : " << f->aire() << "\n";
102         std::cout << "   Perimetre : " << f->perimetre() << "\n\n";
103     }
104
105     // Calcul de l'aire totale
106     double aireTotale = 0;
107     for (Forme* f : formes) {
108         aireTotale += f->aire();
109     }
110     std::cout << "Aire totale : " << aireTotale << "\n";
111
112     // Nettoyage

```

```

113     for (Forme* f : formes) {
114         delete f;
115     }
116
117     return 0;
118 }

```

6.2 Exemple 2 : Systeme de paiement

```

1  class MoyenPaiement {
2  protected:
3      std::string titulaire;
4
5  public:
6      MoyenPaiement(const std::string& t) : titulaire(t) {}
7      virtual ~MoyenPaiement() = default;
8
9      virtual bool payer(double montant) = 0;
10     virtual void afficherDetails() const = 0;
11 };
12
13 class CarteBancaire : public MoyenPaiement {
14 private:
15     std::string numero;
16     double solde;
17
18 public:
19     CarteBancaire(const std::string& t, const std::string& n, double s)
20         : MoyenPaiement(t), numero(n), solde(s) {}
21
22     bool payer(double montant) override {
23         if (solde >= montant) {
24             solde -= montant;
25             std::cout << "Paiement de " << montant
26                 << " euros par carte\n";
27             return true;
28         }
29         std::cout << "Solde insuffisant !\n";
30         return false;
31     }
32
33     void afficherDetails() const override {
34         std::cout << "Carte de " << titulaire
35             << " (solde: " << solde << " euros)\n";
36     }
37 };
38
39 class PayPal : public MoyenPaiement {
40 private:
41     std::string email;
42     double solde;
43
44 public:
45     PayPal(const std::string& t, const std::string& e, double s)
46         : MoyenPaiement(t), email(e), solde(s) {}
47
48     bool payer(double montant) override {

```

```

49     if (solde >= montant) {
50         solde -= montant;
51         std::cout << "Paieement de " << montant
52                 << " euros via PayPal\n";
53         return true;
54     }
55     std::cout << "Solde PayPal insuffisant !\n";
56     return false;
57 }
58
59 void afficherDetails() const override {
60     std::cout << "PayPal de " << titulaire
61             << " (" << email << ", solde: "
62             << solde << " euros)\n";
63 }
64 };
65
66 class Espèces : public MoyenPaieement {
67 private:
68     double montantDisponible;
69
70 public:
71     Espèces(const std::string& t, double m)
72         : MoyenPaieement(t), montantDisponible(m) {}
73
74     bool payer(double montant) override {
75         if (montantDisponible >= montant) {
76             montantDisponible -= montant;
77             std::cout << "Paieement de " << montant
78                     << " euros en especes\n";
79             return true;
80         }
81         std::cout << "Pas assez d'especes !\n";
82         return false;
83     }
84
85     void afficherDetails() const override {
86         std::cout << "Especes (" << montantDisponible
87                 << " euros disponibles)\n";
88     }
89 };
90
91 int main() {
92     CarteBancaire carte("Alice", "1234-5678", 100.0);
93     PayPal paypal("Bob", "bob@email.com", 50.0);
94     Espèces especes("Charlie", 30.0);
95
96     std::vector<MoyenPaieement*> moyens = {&carte, &paypal, &especes};
97
98     // Affichage des moyens de paiement
99     for (auto* m : moyens) {
100         m->afficherDetails();
101     }
102
103     std::cout << "\n=== Paiements ===\n";
104     carte.payer(25.0);
105     paypal.payer(60.0); // Echoue
106     especes.payer(20.0);

```

```
107  
108     return 0;  
109 }
```

7 Exercices pratiques

7.1 Exercice 1 : Hierarchie d'employes (facile)

Enonce

Creez une hierarchie de classes pour des employes :

- Classe de base `Employe` avec nom, salaire de base
- Classe `Manager` qui herite et a un bonus
- Classe `Developpeur` qui herite et a un langage prefere
- Methode virtuelle `calculerSalaire()` qui retourne le salaire total
- Methode virtuelle `afficher()` qui affiche les infos

Utilisation attendue :

```

1 Manager m("Alice", 3000, 500);
2 Developpeur d("Bob", 2500, "C++");
3
4 std::vector<Employe*> equipe = {&m, &d};
5 for (auto* e : equipe) {
6     e->afficher();
7     std::cout << "Salaire total : "
8               << e->calculerSalaire() << "\n\n";
9 }
```

Solution :

```

1 class Employe {
2 protected:
3     std::string nom;
4     double salaireBase;
5
6 public:
7     Employe(const std::string& n, double s)
8         : nom(n), salaireBase(s) {}
9
10    virtual ~Employe() = default;
11
12    virtual double calculerSalaire() const {
13        return salaireBase;
14    }
15
16    virtual void afficher() const {
17        std::cout << "Employe : " << nom << "\n";
18        std::cout << "Salaire de base : " << salaireBase << "\n";
19    }
20 };
21
22 class Manager : public Employe {
23 private:
24     double bonus;
25
26 public:
27     Manager(const std::string& n, double s, double b)
28         : Employe(n, s), bonus(b) {}
29
30     double calculerSalaire() const override {
```

```

31         return salaireBase + bonus;
32     }
33
34     void afficher() const override {
35         std::cout << "Manager : " << nom << "\n";
36         std::cout << "Salaire de base : " << salaireBase << "\n";
37         std::cout << "Bonus : " << bonus << "\n";
38     }
39 };
40
41 class Developpeur : public Employe {
42 private:
43     std::string langage;
44
45 public:
46     Developpeur(const std::string& n, double s, const std::string& l)
47         : Employe(n, s), langage(l) {}
48
49     void afficher() const override {
50         std::cout << "Developpeur : " << nom << "\n";
51         std::cout << "Salaire : " << salaireBase << "\n";
52         std::cout << "Langage prefere : " << langage << "\n";
53     }
54 };

```

7.2 Exercice 2 : Zoo virtuel (moyen)

Enonce

Creez un systeme de zoo avec :

- Classe de base `Animal` avec nom, age, cri()
- Classes derivees : `Lion`, `Elephant`, `Singe`
- Chaque animal a son propre cri
- Methode `sePresenter()` qui affiche nom, age, et fait un cri
- Une classe `Zoo` qui contient un vector d'animaux
- Methode `faireTousLesCris()` dans `Zoo`

Solution partielle :

```

1 class Animal {
2 protected:
3     std::string nom;
4     int age;
5
6 public:
7     Animal(const std::string& n, int a) : nom(n), age(a) {}
8     virtual ~Animal() = default;
9
10    virtual void cri() const = 0; // Virtuelle pure
11
12    void sePresenter() const {
13        std::cout << nom << ", " << age << " ans : ";
14        cri();
15    }
16 };

```

```
17
18 class Lion : public Animal {
19 public:
20     Lion(const std::string& n, int a) : Animal(n, a) {}
21
22     void cri() const override {
23         std::cout << "ROARRR !\n";
24     }
25 };
26
27 class Elephant : public Animal {
28 public:
29     Elephant(const std::string& n, int a) : Animal(n, a) {}
30
31     void cri() const override {
32         std::cout << "BARRRIT !\n";
33     }
34 };
35
36 class Singe : public Animal {
37 public:
38     Singe(const std::string& n, int a) : Animal(n, a) {}
39
40     void cri() const override {
41         std::cout << "OOH OOH AH AH !\n";
42     }
43 };
44
45 class Zoo {
46 private:
47     std::vector<Animal*> animaux;
48
49 public:
50     void ajouterAnimal(Animal* a) {
51         animaux.push_back(a);
52     }
53
54     void faireTousLesCris() const {
55         std::cout << "=== Concert du zoo ===\n";
56         for (const auto* a : animaux) {
57             a->sePresenter();
58         }
59     }
60
61     ~Zoo() {
62         for (auto* a : animaux) {
63             delete a;
64         }
65     }
66 };
67
68 int main() {
69     Zoo z;
70     z.ajouterAnimal(new Lion("Simba", 5));
71     z.ajouterAnimal(new Elephant("Dumbo", 10));
72     z.ajouterAnimal(new Singe("George", 3));
73
74     z.faireTousLesCris();
```

```
75  
76     return 0;  
77 }
```

7.3 Exercice 3 : Calculatrice polymorphe (difficile)

Enonce

Creez un systeme de calcul avec :

- Classe abstraite `Operation` avec methode `calculer()`
- Classes derivees : `Addition`, `Soustraction`, `Multiplication`, `Division`
- Chaque operation prend deux nombres dans son constructeur
- `calculer()` retourne le resultat
- `Division` doit lancer une exception si diviseur = 0
- Une classe `Calculatrice` qui execute une liste d'operations

Solution :

```
1  class Operation {  
2  protected:  
3      double a, b;  
4  
5  public:  
6      Operation(double x, double y) : a(x), b(y) {}  
7      virtual ~Operation() = default;  
8  
9      virtual double calculer() const = 0;  
10     virtual std::string getNom() const = 0;  
11 };  
12  
13 class Addition : public Operation {  
14 public:  
15     Addition(double x, double y) : Operation(x, y) {}  
16  
17     double calculer() const override {  
18         return a + b;  
19     }  
20  
21     std::string getNom() const override {  
22         return "Addition";  
23     }  
24 };  
25  
26 class Soustraction : public Operation {  
27 public:  
28     Soustraction(double x, double y) : Operation(x, y) {}  
29  
30     double calculer() const override {  
31         return a - b;  
32     }  
33  
34     std::string getNom() const override {  
35         return "Soustraction";  
36     }  
37 };
```

```
38
39 class Multiplication : public Operation {
40 public:
41     Multiplication(double x, double y) : Operation(x, y) {}
42
43     double calculer() const override {
44         return a * b;
45     }
46
47     std::string getNom() const override {
48         return "Multiplication";
49     }
50 };
51
52 class Division : public Operation {
53 public:
54     Division(double x, double y) : Operation(x, y) {}
55
56     double calculer() const override {
57         if (b == 0) {
58             throw std::runtime_error("Division par zero !");
59         }
60         return a / b;
61     }
62
63     std::string getNom() const override {
64         return "Division";
65     }
66 };
67
68 class Calculatrice {
69 private:
70     std::vector<Operation*> operations;
71
72 public:
73     void ajouterOperation(Operation* op) {
74         operations.push_back(op);
75     }
76
77     void executerTout() {
78         for (auto* op : operations) {
79             try {
80                 double resultat = op->calculer();
81                 std::cout << op->getNom() << " : "
82                     << resultat << "\n";
83             } catch (const std::exception& e) {
84                 std::cout << op->getNom() << " : ERREUR - "
85                     << e.what() << "\n";
86             }
87         }
88     }
89
90     ~Calculatrice() {
91         for (auto* op : operations) {
92             delete op;
93         }
94     }
95 };
```

```
96
97 int main() {
98     Calculatrice calc;
99
100     calc.ajouterOperation(new Addition(10, 5));
101     calc.ajouterOperation(new Soustraction(10, 5));
102     calc.ajouterOperation(new Multiplication(10, 5));
103     calc.ajouterOperation(new Division(10, 5));
104     calc.ajouterOperation(new Division(10, 0)); // Erreur
105
106     calc.executerTout();
107
108     return 0;
109 }
```

8 Pieges courants

8.1 Pieu 1 : Oublier virtual sur le destructeur

Deja vu dans la section precedente, mais ca vaut la peine de repeter :

TOUJOURS mettre virtual sur le destructeur !

```
1 class Base {
2 public:
3     virtual ~Base() = default; // <- OBLIGATOIRE !
4 };
```

Sinon : fuites memoire garanties quand vous deletez via un pointeur de base.

8.2 Pieu 2 : Slicing (decoupe)

```
1 class Animal {
2 public:
3     virtual void parler() { std::cout << "...\\n"; }
4 };
5
6 class Chien : public Animal {
7 private:
8     std::string race;
9 public:
10    Chien(const std::string& r) : race(r) {}
11    void parler() override { std::cout << "Ouaf !\\n"; }
12 };
13
14 void fonction(Animal a) { // Passage par VALEUR !
15     a.parler();
16 }
17
18 int main() {
19     Chien rex("Labrador");
20     fonction(rex); // Affiche : ...
21     // PROBLEME : rex est "decoupe" en Animal !
22     // La partie Chien est perdue !
23 }
```

Solution

Passez TOUJOURS par reference ou pointeur pour le polymorphisme !

```
1 void fonction(Animal& a) { // Reference !
2     a.parler();
3 }
4
5 // Ou
6 void fonction(Animal* a) { // Pointeur !
7     a->parler();
8 }
```

8.3 Pieu 3 : Appeler une fonction virtuelle dans le constructeur

```
1 class Base {
2 public:
3     Base() {
4         init(); // Appel d'une fonction virtuelle
5     }
6     virtual void init() {
7         std::cout << "Base::init()\n";
8     }
9 };
10
11 class Derivee : public Base {
12 private:
13     int* data;
14 public:
15     Derivee() {
16         data = new int[10];
17     }
18
19     void init() override {
20         std::cout << "Derivee::init()\n";
21         data[0] = 42; // CRASH potentiel !
22     }
23 };
24
25 int main() {
26     Derivee d;
27     // Affiche : Base::init()
28     // PAS Derivee::init() !
29 }
```

Pourquoi ca ne marche pas ?

Dans le constructeur de Base, l'objet Derivee n'est PAS ENCORE construit !
Le polymorphisme ne fonctionne pas dans les constructeurs. C++ appelle toujours la version de la classe en cours de construction.

Solution : Evitez d'appeler des fonctions virtuelles dans les constructeurs.

9 Quiz de validation

9.1 Questions

1. Qu'est-ce que l'heritage et quand l'utiliser ?
2. Quelle est la difference entre **public**, **protected** et **private** ?
3. Qu'est-ce que le polymorphisme ?
4. Pourquoi mettre **virtual** sur une fonction ?
5. Pourquoi le destructeur d'une classe de base DOIT-il etre virtuel ?
6. Que fait le mot-cle **override** ?
7. Quelle est la difference entre heritage et composition ?
8. Dans quel ordre les constructeurs sont-ils appeles ?
9. Qu'est-ce que le "slicing" ?
10. Peut-on instancier une classe avec une fonction virtuelle pure ?

9.2 Reponses

1. L'heritage permet a une classe (derivee) de recuperer les membres et methodes d'une autre classe (base). Utiliser quand "X EST-UN Y".
2. **public** : accessible partout. **protected** : accessible dans la classe et les classes derivees. **private** : accessible uniquement dans la classe.
3. Le polymorphisme permet qu'un meme appel de fonction ait des comportements differents selon le type reel de l'objet. Grace a **virtual**.
4. Pour que C++ regarde le type reel de l'objet au moment de l'execution (polymorphisme), pas le type du pointeur.
5. Sinon, seul le destructeur de la base est appele quand on delete via un pointeur de base. Fuite memoire garantie !
6. **override** demande au compilateur de verifier qu'on redefinit vraiment une fonction virtuelle existante. Detecte les erreurs de signature.
7. Heritage = "EST-UN" (Chien EST-UN Animal). Composition = "A-UN" (Voiture A-UN Moteur). Preferer la composition quand c'est possible.
8. De la base vers la derivee (fondations avant murs). Destruction dans l'ordre inverse.
9. Le slicing se produit quand on passe un objet derive par valeur a une fonction qui attend la base. La partie derivee est "decoupee".
10. Non ! Une classe avec au moins une fonction virtuelle pure est abstraite et ne peut pas etre instanciee.

10 Pour aller plus loin

10.1 Heritage multiple

C++ permet d'hériter de plusieurs classes à la fois :

```

1 class Terrestre {
2 public:
3     void marcher() { std::cout << "Je marche\n"; }
4 };
5
6 class Aquatique {
7 public:
8     void nager() { std::cout << "Je nage\n"; }
9 };
10
11 class Amphibien : public Terrestre, public Aquatique {
12     // Herite des deux !
13 };
14
15 int main() {
16     Amphibien grenouille;
17     grenouille.marcher();
18     grenouille.nager();
19 }

```

Attention : L'héritage multiple peut devenir complexe (probleme du diamant). A utiliser avec precaution.

10.2 Classes abstraites pures

Une classe avec AU MOINS une fonction virtuelle pure ne peut pas être instanciée :

```

1 class Interface {
2 public:
3     virtual void methode() = 0; // Virtuelle pure
4     virtual ~Interface() = default;
5 };
6
7 // Interface i; // ERREUR ! Interface est abstraite
8
9 class Implementation : public Interface {
10 public:
11     void methode() override {
12         // Implementation concrete
13     }
14 };
15
16 Implementation impl; // OK !

```

Utilite : Définir des interfaces (contrats) que d'autres classes doivent implémenter.

10.3 Exercices bonus

1. **Système de fichiers :** Créez une hiérarchie Fichier/Dossier ou un Dossier peut contenir des Fichiers et d'autres Dossiers.
2. **RPG :** Créez des classes pour un jeu de rôle : Personnage (base), Guerrier, Mage, Archer. Chacun a des attaques différentes.

3. **Instruments** : Classe de base Instrument avec methode jouer(). Classes derivees : Piano, Guitare, Batterie. Orchestre qui fait jouer tous les instruments.

10.4 Ressources supplementaires

- **cppreference.com** : Documentation sur l’heritage et le polymorphisme
- **learncpp.com** : Tutoriels progressifs (chapitres 17-18)
- **Effective C++** de Scott Meyers : Items sur l’heritage (notamment Item 7 sur les destructeurs virtuels)

Fin de la Seance 2

Prochaine seance : Classes Abstraites et Interfaces